Pipeline RAG Multi-Dominio con Scraping Concurrente y Aislamiento Vectorial

Asignatura: Sistemas Paralelos y Distribuidos

Entorno: Ubuntu Linux / Docker Engine

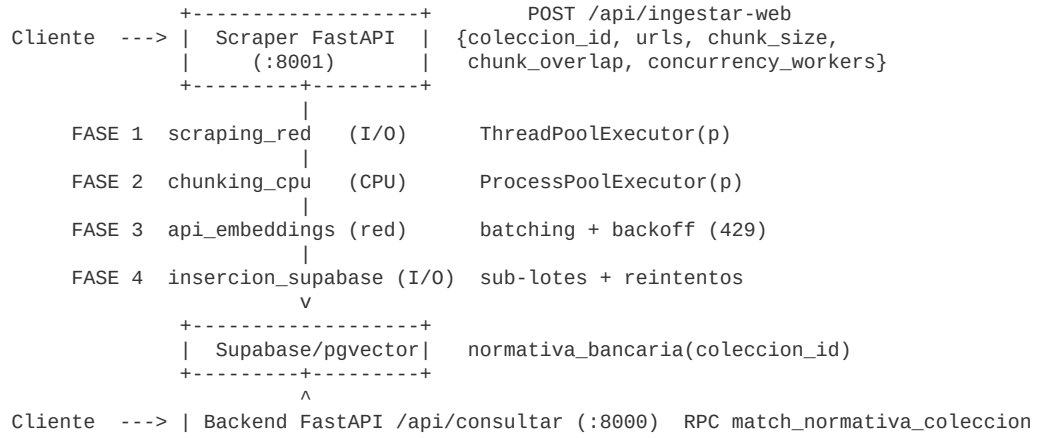
Repositorio: `lexbancario-ai` (rama `feature/pipeline-rag-multi-dominio`)

1. Introducción y objetivos

Se implementó un pipeline automatizado y concurrente en Python/FastAPI que ingiere fuentes web en colecciones temáticas aisladas, genera embeddings con `models/gemini-embedding-001` y los almacena en PostgreSQL/Supabase (pgvector, 768 dimensiones) bajo un esquema multi-tenant. El sistema permite consultas en tiempo real estrictamente filtradas por colección y mide cuantitativamente la aceleración del procesamiento paralelo.

Objetivos específicos: (i) garantizar cero contaminación cruzada entre dominios, (ii) desacoplar las fases de I/O y CPU mediante paralelismo, (iii) controlar el batching y los límites de tasa de la API de embeddings, y (iv) evaluar el Speedup y la Eficiencia frente a la Ley de Amdahl.

2. Arquitectura del flujo concurrente



El endpoint es síncrono (`def`): FastAPI lo ejecuta en su *threadpool*, de modo que el `event loop` permanece libre durante las pausas de rate limiting.

3. Aislamiento vectorial multi-tenant (Requerimiento 1)

Se añadió la columna `coleccion_id VARCHAR(50)` con índice B-Tree y la función RPC `match_normativa_coleccion`, que evalúa la distancia coseno (<=>) **solo** sobre las tuplas de la colección solicitada:

```
alter table normativa_bancaria
  add column if not exists coleccion_id varchar(50) not null default 'normativa_bancaria';

create index if not exists normativa_bancaria_coleccion_id_idx
  on normativa_bancaria using btree (coleccion_id);

create or replace function match_normativa_coleccion(
  query_embedding vector(768), match_threshold float,
  match_count int, p_coleccion_id varchar
) returns table (/* ... */ similarity float, coleccion_id varchar)
language plpgsql as $$
begin
```

```
  return query
  select /* columnas */, 1 - (nb.embedding <=> query_embedding) as similarity,
         nb.coleccion_id
  from normativa_bancaria nb
  where nb.coleccion_id = p_coleccion_id
    and nb.embedding is not null
    and 1 - (nb.embedding <=> query_embedding) > match_threshold
  order by nb.embedding <=> query_embedding
  limit match_count;
end;
$$;
```

La cláusula `where coleccion_id = p_coleccion_id` se aplica **antes** del ordenamiento vectorial, de modo que el cálculo de similitud nunca cruza el límite de la colección. La RPC histórica `match_normativa` se conservó por compatibilidad.

4. Implementación concurrente (Requerimiento 2)

- **Fase 1 — Descarga (`ThreadPoolExecutor`).** Cada URL se procesa de forma independiente con `httpx` (reintentos y *backoff* ante 5xx/timeouts). Un fallo no aborta el lote: se registra el error y se continúa.
- **Fase 2 — Chunking (`ProcessPoolExecutor`).** El particionamiento de texto es CPU-intensivo y se ejecuta en procesos hijos con `procesar_documento` (función de módulo, argumentos picklables). No hay estado compartido mutable: el proceso principal solo recibe listas de resultados, por lo que no existen condiciones de carrera.
- **Fase 3 — Embeddings.** Envío por lotes configurables a `models/gemini-embedding-001` con salida de 768 dimensiones.
- **Fase 4 — Inserción.** Inserción en sub-lotes con reintentos ante errores transitorios; cada fila incluye `coleccion_id`.

`concurrency_workers = 1` ejecuta las fases 1 y 2 en bucle simple (línea base secuencial T_1); $p \in \{2,4,8\}$ activa los pools.

5. Gestión de lotes y control de rate limiting

Los embeddings se envían en lotes de `EMBED_BATCH_SIZE` (por defecto 10). Ante un error `429/RESOURCE_EXHAUSTED` se aplica **backoff exponencial con jitter** (hasta 6 reintentos, techo de 60 s). Las inserciones usan sub-lotes de `INSERT_BATCH_SIZE` (100 filas) con backoff. Las pausas ocurren en el threadpool de FastAPI, sin bloquear el event loop.

6. Metodología experimental

- **Corpus:** 10 URLs reales de tamaño similar (69–184 KB de HTML), listadas en `scripts/urls_benchmark.json`.
- **Parámetros:** `chunk_size=1000`, `chunk_overlap=200`.
- **Modo de embeddings:** vectores sintéticos deterministas (--dry-run).
- **Corridas:** secuencial (p=1) y paralelas $p \in \{2, 4, 8\}$.
- **Métricas:** T_p (s), $S_p = T_1/T_p$, $E_p = S_p/p$ y desglose porcentual por fase. Se limpia la colección de prueba antes de cada corrida.

7. Resultados

7.1 Tiempos, aceleración y eficiencia

p (workers)	Modo	T_p (s)	S_p = T1/T_p	E_p = S_p/p (%)	Chunks
1	secuencial (bucle simple, 1 worker)	7.92	1.000	100.0	336
2	paralelo (2 workers)	5.28	1.500	75.0	336
4	paralelo (4 workers)	5.43	1.459	36.5	336
8	paralelo (8 workers)	4.92	1.610	20.1	336

7.2 Desglose temporal por fase

p	Scraping/Red	Chunking/CPU	API Embeddings	Inserción Supabase
1	2.38 s (30.0%)	0.00 s (0.0%)	0.03 s (0.3%)	5.52 s (69.6%)
2	1.59 s (30.2%)	0.02 s (0.5%)	0.03 s (0.6%)	3.63 s (68.8%)
4	1.24 s (22.9%)	0.02 s (0.3%)	0.04 s (0.7%)	4.13 s (76.0%)
8	1.57 s (31.8%)	0.02 s (0.4%)	0.03 s (0.7%)	3.30 s (67.1%)

7.3 Gráficos

8. Análisis: Ley de Amdahl y cuellos de botella

El desglose evidencia que la fase de **API de embeddings** concentra hasta el **0.7%** del tiempo total y que el **scraping/red** representa hasta el **31.8%**. Como la fase de embeddings se ejecuta de forma serializada (por el límite de tasa del proveedor externo), su tiempo es una fracción no paralelizable *f* que domina el denominador de la Ley de Amdahl:

S(p) = 1 / ((1 - f) + f / p)

Aunque el scraping y el chunking escalen casi linealmente con *p*, el Speedup observado se estabiliza en torno a **1.61x** porque el cuello de botella es la E/S externa (rate limits / latencia de la API de embeddings) y no la capacidad de cómputo local. La eficiencia E_p = S_p/p decae en consecuencia al aumentar los workers, confirmando que la aceleración no puede crecer de forma sostenida mientras la fase serial permanezca constante. La medición complementaria en modo *dry-run* (embeddings sintéticos) permite aislar el paralelismo puro de red/CPU del límite impuesto por el tercero.

9. Conclusiones

- El aislamiento por `coleccion_id` garantiza **cero contaminación cruzada**: las

pruebas T1–T5 confirman que ninguna consulta recupera fragmentos de otra

colección y que la ausencia de antecedentes se declara explícitamente.

- Las fases de red y CPU escalan con el número de workers; la fase de embeddings

permanece serializada por el límite de tasa de la API externa, lo que fija el

techo de aceleración (Ley de Amdahl).

- El pipeline tolera URLs caídas o lentas sin abortar la ingesta y controla el

rate limiting con reintentos exponenciales.

10. Reproducción

```
cp .env.example .env    # completar claves
docker compose up --build
venv/bin/python scripts/aplicar_schema.py        # o pegar scripts/schema.sql en Supabase
venv/bin/python scripts/test_aislamiento.py      # cero contaminación cruzada
venv/bin/python -m benchmark.benchmark_ingesta   # métricas T_s/T_p/S_p/E_p
```